Predicting Stock Prices

with Regression Algorithms

Machine Learning Practice With Python

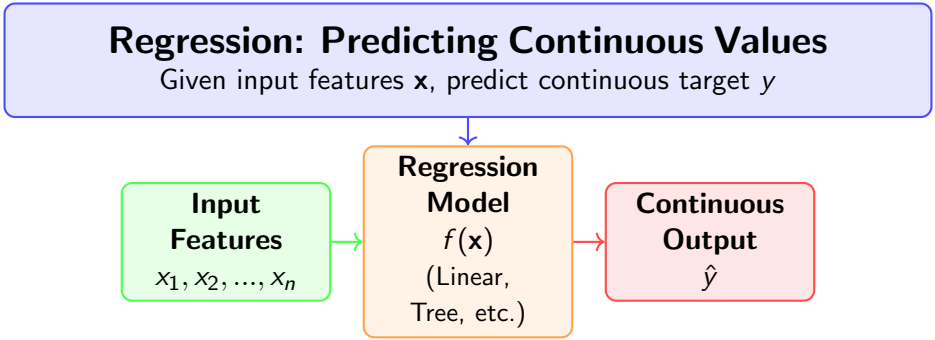
Siksha 'O' Anushandhan, ITER

Center for Data Science

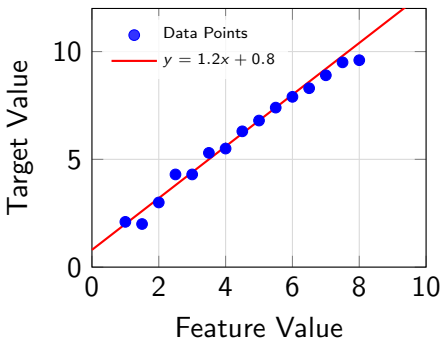
Table of Contents

1. Stock Price Prediction by Linear Regression
2. Linear regression by TensorFlow
3. Prediction by Decision Tree
4. Prediction by Random Forest
5. Best parameters of each model

What is a Regression Problem?



Linear Regression:Finding the Best Fit L



Linear Regression

Linear Model:

$$y = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + ... + \theta_n x_n = \theta^T X$$

Cost Function (MSE):

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^{m} (\hat{y}^{(i)} - y^{(i)})^2 + \alpha P(\theta)$$

where, $P(\theta)$ is the penalty function and α is the regularization parameter.

Goal: Minimize $J(\theta)$

Linear Regression Visualization

Optimized w

Normal Equation

$$\theta^* = (X^T X + n\alpha I)^{-1} X^T Y \qquad \text{(Prove it)}$$

Gradient descent

$$\theta^{(i+1)} = \theta^{(i)} - \eta \frac{\partial J(\theta)}{\partial \theta}$$

$$= \left[I - \eta \left(\frac{1}{n} X^T X + \alpha I \right) \right] \theta^{(i)} + \frac{\eta}{n} X^T Y \qquad \text{(Prove it)}$$

Understanding Stocks and Stock Markets

What is a Stock?

- Represents ownership in a corporation
- Single share = claim on fractional assets & earnings
- Traded on exchanges (NYSE, NASDAQ)
- Prices fluctuate due to **supply and demand**

Two Approaches to Stock Analysis

FUNDAMENTAL ANALYSIS

Focuses on underlying factors:

Macro-economic conditions
Industry conditions
Company's financial health
Management quality
Competitor analysis

Analyzes intrinsic company value

VS

TECHNICAL ANALYSIS

Studies past trading activity:

Price movements
Trading volume
Market data patterns
Statistical indicators

Machine Learning techniques *are significant part of this approach*

↓

Our Approach

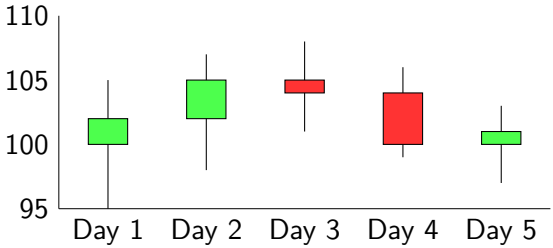
Key Trading Indicators (Daily Data)

Candlestick Components:

- **Green**: Close > Open (bullish)
- **Red**: Close < Open (bearish)
- **Wicks**: High and Low extremes

5 Key Daily Indicators:

- **Open (O)**: Starting price
- **Close (C)**: Final price
- **High (H)**: Highest price
- **Low (L)**: Lowest price
- **Volume**: Shares traded



Date	Open	High	Low	Close	Volume
Day 1	100	105	95	102	2000
Day 2	102	107	97	105	2100
Day 3	105	108	100	103	1900
Day 4	103	105	97	100	2200
Day 5	99	102	96	100	2000

Feature Engineering for Stock Prediction

Original Features

Open prices
Historical high
Historical low
Historical volume
Previous close

Problem

Original features alone may lead to **UNDERFITTING**

Solution

Feature Engineering

Design & construct specific features from raw data

RESULT: $(p + 5)$ Total Features

5 Original Features + p Engineered Features

Requires **domain knowledge** and can be **time-consuming**

Engineered features

Moving Average of last n days closing price on i-th day:

$$AvgPrice_n^i = \frac{Close_{(i-1)} + Close_{(i-2)} + \cdots + Close_{(i-n)}}{n}$$

Ratio of moving average close price of last m and n days in i-th day:

$$AvgPriceRatio_{m,n}^i = \frac{AvgPrice_m^i}{AvgPrice_n^i}$$

Engineered features

Moving Average of last n days closing price on i-th day:

$$AvgPrice_n^i = \frac{Close_{(i-1)} + Close_{(i-2)} + \cdots + Close_{(i-n)}}{n}$$

Ratio of moving average close price of last m and n days in i-th day:

$$AvgPriceRatio_{m,n}^i = \frac{AvgPrice_m^i}{AvgPrice_n^i}$$

Exercise:Write formula to find

- Moving average standard deviation of last n days.
- Moving average volume of last n days.
- Return of last n days.

Exercise:Write formula to find

- Ratio of moving average standard deviation of last m and n days.
- Ratio of moving average volume of last m and n days.
- Ratio of return of last m and n days.

Engineered features

- **Moving Average**: Past 5 (week), 21 (month), 252 (year) days
- **Moving Average Ratio**: week/month, week/year, month/year

Price-Based

- Closed price
- Ratio of closed price
- Standard deviation closed price
- Ratio of standard deviation closed price

Volume-Based

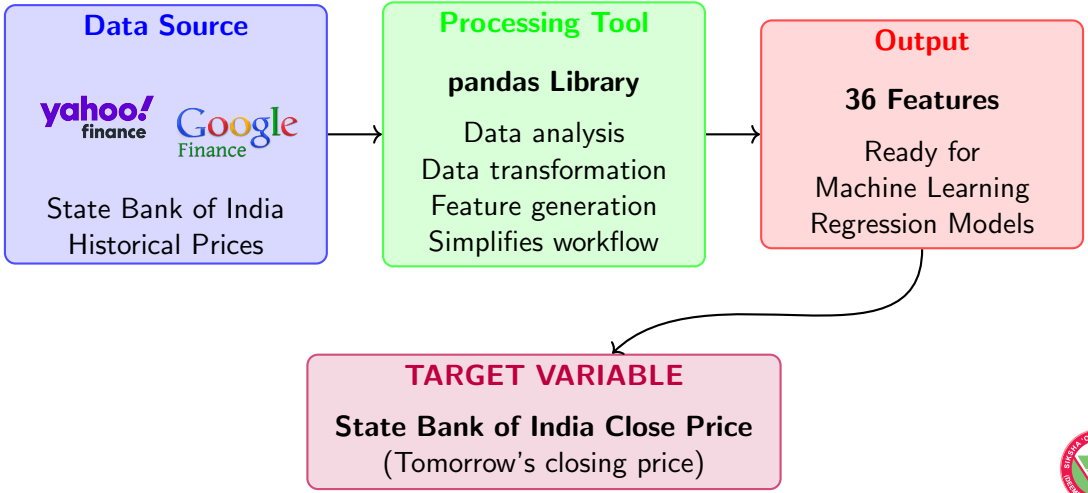
- Volume
- Ratio of volume
- Standard deviation volume
- Ratio of standard deviation volume

Return-Based

- Return (including daily)
- Moving average of return

Key Principle: Exclude same-day performance data (current high, low, volume) as they cannot be predicted

Implementation Tools & Data Sources



Sample SBI historical Data

```
1 import yfinance as yf
2 df = yf.Ticker("SBIN.NS").history(period="max")
3 df.to_csv("SBI_historical_data.csv")
4 df.head()
```

Date	Open	High	Low	Close	Volume	Dividends	Stock Splits
01-01-2024 15:30	642.20	646.90	638.00	641.35	8295548	0	0
01-02-2024 15:30	641.35	648.00	633.85	639.45	15164482	0	0
01-03-2024 15:30	639.35	648.00	635.80	643.45	14571772	0	0
01-04-2024 15:30	642.50	646.40	638.65	642.75	13883388	0	0
01-05-2024 15:30	645.00	651.75	637.75	641.95	15984585	0	0

Plot stock close price over time

Plot stocks close price over time

Plot stock close price over time

Plot stocks close price over time

```
1 import matplotlib.pyplot as plt
2 plt.figure(figsize=(12,6))
3 plt.plot(df['Close'], label='Close Price', color='blue')
4 plt.title('SBI Stock Close Price Over Time')
5 plt.xlabel('Date')
6 plt.ylabel('Close Price')
```

Feature Engineering

Define a function to find **moving average** and **moving average ratio** of closed price of last **week, month**, and **year**.

Feature Engineering

Define a function to find **moving average** and **moving average ratio** of closed price of last **week, month**, and **year**.

```
def moving_average_and_ratio_close(data):
      data['MA_week'] = df['Close'].rolling(window=5).mean().shift(1)
      data['MA_month'] = df['Close'].rolling(window=21).mean().shift(1)
      data['MA_year'] = df['Close'].rolling(window=252).mean().shift(1)
      data['MA_ratio_week_month'] = data['MA_week'] / data['MA_month']
      data['MA_ratio_week_year'] = data['MA_week'] / data['MA_year']
      data['MA_ratio_month_year'] = data['MA_month'] / data['MA_year']
      return data
```

```
df=moving_average_and_ratio_close(df)
print(df.tail(5))
```

N.B.: You can create functions for MA of last *n* days, and ratio of last *m* and *n* days, and call one by one.

Feature Engineering

Exercise:

1. Define a function to calculate the **moving average** and the **ratio of the moving average** to the standard deviation of the closing prices for the last **week, month**, and **year**.
2. Define a function to calculate **moving average** and **ratio of moving average** of volume of the last **week, month**, and **year**.
3. Define a function to calculate the **moving average** and the **ratio of the moving average** to the standard deviation of the volume for the last **week, month**, and **year**.
4. Define a function to calculate return of **daily, week, month**, and **year**. Also, calculate **moving average** of return of **week, month**, and **year**.

Stock Price Prediction by Linear Regression

```
SGDRegressor(loss='squared_error', penalty='l2', alpha=0.0001,
l1_ratio=0.15, fit_intercept=True, max_iter=1000, tol=0.001,
shuffle=True, verbose=0, epsilon=0.1, random_state=None,
learning_rate='invscaling', eta0=0.01, power_t=0.25,
early_stopping=False, validation_fraction=0.1, n_iter_no_change=5,
warm_start=False, average=False)
```

Stock Price Prediction by Linear Regression

```
SGDRegressor(loss='squared_error', penalty='l2', alpha=0.0001,
l1_ratio=0.15, fit_intercept=True, max_iter=1000, tol=0.001,
shuffle=True, verbose=0, epsilon=0.1, random_state=None,
learning_rate='invscaling', eta0=0.01, power_t=0.25,
early_stopping=False, validation_fraction=0.1, n_iter_no_change=5,
warm_start=False, average=False)
```

Build Linear regression model by SGDRegressor

Stock Price Prediction by Linear Regression

```
SGDRegressor(loss='squared_error', penalty='l2', alpha=0.0001,
l1_ratio=0.15, fit_intercept=True, max_iter=1000, tol=0.001,
shuffle=True, verbose=0, epsilon=0.1, random_state=None,
learning_rate='invscaling', eta0=0.01, power_t=0.25,
early_stopping=False, validation_fraction=0.1, n_iter_no_change=5,
warm_start=False, average=False)
```

Build Linear regression model by SGDRegressor

```
1 from sklearn.linear_model import SGDRegressor
2 sgd = SGDRegressor(loss='squared_error', penalty='l2',
        max_iter=1000, alpha= 0.1, learning_rate='constant',
        eta0=0.01, random_state=42)
```

Train Test split data

Before splitting the data, note that all engineered features have `NaN` values. Therefore, remove all `NaN` values before splitting

Train Test split data

Before splitting the data, note that all engineered features have `NaN` values. Therefore, remove all `NaN` values before splitting

- Remove `NaN` values.
- Consider 'Close' column as target column, and remaining all except 'Dividends', 'Stock Splits' as features columns.
- Consider first 80% data as train and last 20% data as test data (as stock data is depend on dates).

Train Test split data

Before splitting the data, note that all engineered features have `NaN` values. Therefore, remove all `NaN` values before splitting

- Remove `NaN` values.
- Consider 'Close' column as target column, and remaining all except 'Dividends', 'Stock Splits' as features columns.
- Consider first 80% data as train and last 20% data as test data (as stock data is depend on dates).

```
df = df.dropna(axis=0)
X = df.drop(columns=['Close', 'Dividends', 'Stock Splits'], axis=1)
Y = df['Close']
X_train = X[:int(0.8*len(X))]
X_test = X[int(0.8*len(X)):]
Y_train = Y[:int(0.8*len(Y))]
Y_test = Y[int(0.8*len(Y)):]
```

Train model and predict

- Train the model.
- predict for test data.
- calculate mean square error.
- Plot test and predicted data.

Train model and predict

- Train the model.
- predict for test data.
- calculate mean square error.
- Plot test and predicted data.

```
sgd.fit(X_train, Y_train)
prediction = sgd.predict(X_test)
```

```
from sklearn.metrics import mean_squared_error
print('error:', mean_squared_error(Y_test, prediction))
```

```
import matplotlib.pyplot as plt
plt.figure(figsize=(12,6))
plt.plot(Y_test.values, label='Actual Close Prices', color='blue')
plt.plot(prediction, label='Predicted Close Prices', color='red')
plt.title('SBI Stock Price Prediction')
plt.xlabel('Time')
plt.ylabel('Stock Price')
plt.legend()
plt.show()
```

Exercise:

- Check accuracy, F1-score, Recall Precission for the following scenario in `SGDRegressor`
  - `learning_rate = 'invscaling', 'optimal', 'adaptive'`
  - Use early stopping.
- Use `SGDRegressor` by mini-batch gradient descent method.

Linear regression by TensorFlow

```
import tensorflow as tf
layer0 = tf.keras.layers.Dense(units=1,input_shape=[X_train.shape[1]])
model = tf.keras.Sequential([layer0])
# model = tf.keras.Sequential([
#       tf.keras.layers.Dense(16, activation='relu',
            input_shape=[X_train.shape[1]]),
#       tf.keras.layers.Dense(1)])
```

```
model.compile(optimizer='adam', loss='mse')
model.fit(X_train, Y_train, epochs=50, batch_size=32,
          validation_split=0.2)
```

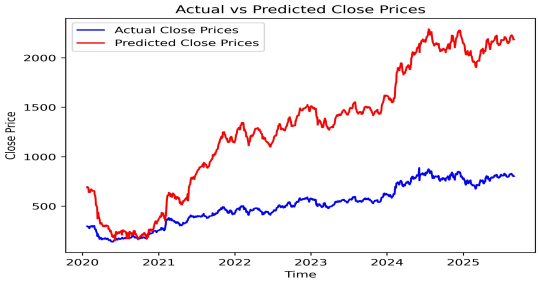
```
prediction = model.predict(X_test)
print('error:', mean_squared_error(Y_test, prediction))
```

Exercise:

Create a neural network with two hidden layer consisting 16 neurons each, and predict.

Linear regression by TensorFlow

```
plt.Figure(figsize=(12,6))
plt.plot(Y_test.values, label='Actual Close Prices', color='blue')
plt.plot(prediction, label='Predicted Close Prices', color='red')
plt.title('Actual vs Predicted Close Prices')
plt.xlabel('Time')
plt.ylabel('Close Price')
plt.legend()
plt.show()
```



Prediction by Decision Tree

- Define Decision Tree model for regression, and predict stock price for test data.
- Print mean square error. Also plot actual and prediction result.

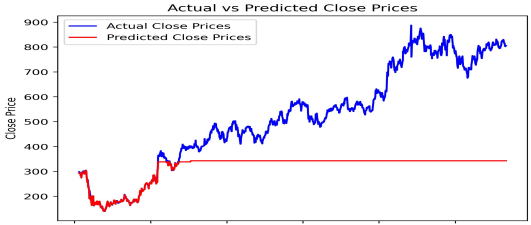
Prediction by Decision Tree

- Define Decision Tree model for regression, and predict stock price for test data.
- Print mean square error. Also plot actual and prediction result.

```
1 from sklearn.tree import DecisionTreeRegressor
2 regressor = DecisionTreeRegressor(max_depth=10, min_samples_split=3,
          random_state=42)
3 regressor.fit(X_train, Y_train)
```

```
1 predictions = regressor.predict(X_test)
2 print('error:', mean_squared_error(Y_test, predictions))
```

```
error: 68445.19569
```



Stock Prediction by Random Forest model

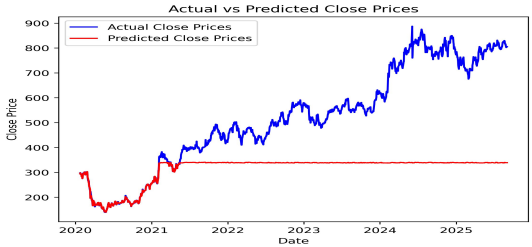
Predict Stock price by Random Forest and plot actual vs prediction

Stock Prediction by Random Forest model

Predict Stock price by Random Forest and plot actual vs prediction

```
1 from sklearn.ensemble import RandomForestRegressor
2 regressor = RandomForestRegressor(n_estimators=100, max_depth=10,
        min_samples_split=3, random_state=42)
3 regressor.fit(X_train, Y_train)
4 predictions = regressor.predict(X_test)
5 print('error:', mean_squared_error(Y_test, predictions))
```

```
error: 69884.8348
```



Exercise:

- Normalize data.
- Find best parameters for `SGDRegressor`, `DecisionTreeRegressor`, `RandomForestRegressor` by `GridSearchCV` method.

Exercise:

- Normalize data.
- Find best parameters for `SGDRegressor`, `DecisionTreeRegressor`, `RandomForestRegressor` by `GridSearchCV` method.

Here usual cross-validation will not work as stock data is a time series data. Therefor, for cross-validation, you have to make sure that training and test data must follow time series. For this, you may use `TimeSeriesSplit` module.

```
1 from sklearn.model_selection import TimeSeriesSplit
2 tscv = TimeSeriesSplit(n_splits=5)
3 grid_search = GridSearchCV(sgd, param_grid, cv=tscv,
          scoring='neg_mean_squared_error')
4 grid_search.fit(X_train_scaled, Y_train)
5 print("Best parameters:", grid_search.best_params_)
6 best_sgd = grid_search.best_estimator_
7 predictions_sgd = best_sgd.predict(X_test_scaled)
```

Prediction by SGDRegressor